

Habitiq — Complete Technical Audit

Project: Habitiq (repo at `C:\garbage`)

Audit date: 9 September 2026

Scope: Read-only analysis of the existing codebase. No code was modified.

Source of truth: Files, configuration, Firestore rules, Android sources, and package manifests in this repository — not marketing docs when they conflict with code.

Companion scorecard (Cursor canvas): `workspace canvases/habitiq-codebase-audit.canvas.tsx` .

1. Executive project status

What it is. Habitiq is a **shared-flat operations app**: chore rotation, swaps, expenses/settlements, recurring bills, members, and (on Android) a **Discover** marketplace for vacancies / flatmates.

Problem it solves (from product copy + code). Replace WhatsApp-group chore fights and Splitwise-only money tracking with one real-time household system.

Current product. Two clients on the same Firebase project (`garbage-f79f7` in `next.config.ts` rewrites and `android/README.md`);

- **Web:** Next.js 16 App Router PWA (`package.json` name `habitiq` , version `0.1.0`).
- **Android:** Kotlin Compose (`android/` , package `habitiq.app`).

There is **no custom REST/backend service**. Firestore security rules + client SDKs *are* the backend.

Already implemented (evidence). Rotation engine (`lib/rotationEngine.ts`), large Zustand store with live `onSnapshot` listeners (`store/useFlatStore.ts`), expenses/bills/settlements/month cycles, swap + join-request flows, Google + email auth (`store/useAuthStore.ts`), trial/coupon UI (`hooks/useSubscription.ts` , `lib/couponService.ts`), PWA/SEO (`app/layout.tsx` , `middleware.ts` → `habitiq.app`), Android Discover + FCM token save.

Functional today (if Firebase keys exist and indexes exist in the cloud). Sign in, create/join flat, tasks, swaps, expenses, bills, members, insights from activity log, leave/kick/transfer admin.

Partially functional. Subscriptions (client overlay only), Discover (Android UI + rules; **web types/modules missing**), FCM (token stored, no send pipeline in this repo), password reset (**Android only**).

UI / mock. If `NEXT_PUBLIC_FIREBASE_API_KEY` is missing, `lib/firebase.ts` runs **Local Mock Mode** with seeded members/tasks/bills (`store/useFlatStore.ts` `MOCK_*`). Playwright is written against that (`playwright.config.ts`).

Completely missing. Stripe/Razorpay, WhatsApp API, Cloud Functions, queues/cron, web Discover screens, web password reset / email verification, GitHub Actions, committed Firestore composite indexes.

Intended final product (from code, not docs). Household OS **plus** Discover (vacancy on `flats/{id}` , `seekerProfiles` , `messages` , `discoveryConnections` in `firestore.rules` and Android `DiscoveryRepository.kt` / `MessagingRepository.kt`).

Stage. Functional MVP for in-flat ops; early / incomplete for Discover and monetization. Docs disagree with each other (`PRODUCT.md` says live trial; `package.json` is `0.1.0` ; other notes mention later versions — treat git/code as source of truth).

Completion estimates (method)

Each number = **implemented in this tree ÷ what this tree is clearly trying to ship**. Not uptime of habitiq.app.
Runtime of production was not verified in the audit session.

Area	%	Basis
Frontend	72%	Most web dashboard pages exist and bind to the store; Android has Home/Discover/Manage/Profile. Minus Discover on web, obsolete routes, missing web auth extras.
Backend	58%	Rules + client services cover household writes. No Functions, no server jobs, trial not in rules.
Database	70%	Rich firestore.rules model. firestore.indexes.json is { "indexes": [] }.
Authentication	68%	Google + email both clients; persist session on web; reset only on Android; no email verify found.
API layer	40%	Direct Firestore. Only HTTP route found: app/api/pwa-icon/[size]/route.tsx.
Integrations	32%	Firebase Auth/Firestore/Analytics + Android FCM token. Payments/WhatsApp/maps-as-product not wired as live APIs in web.
Core business logic	78%	Rotation, splits, bills, swaps are substantial in TS + Kotlin.
Admin/dashboard	75%	Admin task CRUD, kick, join mode, manage-flat, NPS read — all client-side.
Production readiness	42%	Hosting config exists (Vercel rewrites + Netlify toml + 301 middleware) but web imports missing files , empty indexes, no CI, trial bypassable.

2. Complete project blueprint

User
→ Web (Next.js) or Android (Compose)
→ UI (app/*, components/*, android/.../ui)
→ Zustand (web) / ViewModels (Android)
→ Firebase client SDK (no Next server actions for domain data)
→ Firestore + Auth
→ Google OAuth, Analytics, FCM (Android)

Layer	Tech	Files	Working	Incomplete
Frontend web	Next 16, React 19, Tailwind 4, Framer Motion	app/, components/	Marketing + dashboard	Discover UI; /login dead
Frontend Android	Kotlin Compose	android/app/src/main/kotlin/habitiq/app/	Auth, flats, Discover, FCM	Cannot determine

			register	Play-store readiness from code alone
State	Zustand persist habitiq-auth	store/useAuthStore.ts, store/useFlatStore.ts	Listeners when hasKeys	Mock when not
"API"	Firestore SDK	lib/flatService.ts, store writes	Client CRUD	No REST
Business logic	TS/Kotlin	lib/rotationEngine.ts, expenseUtils.ts, settlementUtils.ts, Android lib/	Rotation + money math	Discover ranking Android-only
Database	Cloud Firestore	firestore.rules	Rules authored	Indexes empty
AuthZ	Rules + UI role checks	firestore.rules, layouts	Member/admin split	Trial not in rules
Infra	Next headers/rewrites, middleware 301, netlify.toml	next.config.ts, middleware.ts	Dual-host leftovers	No vercel.json; no .github/

3. Frontend audit

Web pages / routes

Auth routing is **client-only** in `components/AuthProvider.tsx` : no user → `/` (except `/privacy` , `/terms`); user without `flatId` → `/onboarding` ; user with flat on `/` → `/dashboard` . **`/login` and `/join` are not real auth pages.**

Route	Purpose	File	Data
<code>/</code>	Marketing + Google/email/mock login	<code>app/page.tsx</code> + <code>components/ui/navbar.tsx</code>	Static + AuthForm
<code>/login</code>	Obsolete	<code>app/(auth)/login/page.tsx</code> returns null	None
<code>/join</code>	Compat	<code>app/(auth)/join/page.tsx</code>	<code>redirect('/onboardi</code>

/onboarding	Create/join/approval pending	app/onboarding/page.tsx	lib/flatService.ts
/dashboard	Home: tasks, swaps, balances, NPS	app/dashboard/page.tsx	useFlatStore snapshots
/dashboard/insights	Calendar + stats from activity	app/dashboard/insights/page.tsx	activityLog, tasks
/dashboard/analytics	Redirect	app/dashboard/analytics/page.tsx	—
/dashboard/calendar	Redirect	app/dashboard/calendar/page.tsx	—
/dashboard/expenses	Splits + bills	app/dashboard/expenses/page.tsx	expenses, bills, cycle
/dashboard/tasks	Admin/member task mgmt	app/dashboard/tasks/page.tsx	tasks
/dashboard/swaps	Swap inbox	app/dashboard/swaps/page.tsx	swapRequests
/dashboard/members	Roster, kick, join requests	app/dashboard/members/page.tsx	members, joinRequest
/dashboard/members/[uid]	Member stats	app/dashboard/members/[uid]/page.tsx	members + activity
/dashboard/profile	Account, flats, theme, leave	app/dashboard/profile/page.tsx	auth + members
/dashboard/settings	Overlaps profile (theme, leave)	app/dashboard/settings/page.tsx	same
/dashboard/manage-flat	Invite, rename, NPS	app/dashboard/manage-flat/page.tsx	flat + NPS
/dashboard/activity	Activity filters	app/dashboard/activity/page.tsx	activityLog
/dashboard/about	About	app/dashboard/about/page.tsx	static
/privacy, /terms	Legal	app/privacy/page.tsx, app/terms/page.tsx	static
/api/pwa-icon/[size]	OG/PWA PNG	app/api/pwa-icon/[size]/route.tsx	/habitiq-icon.png

`app/robots.ts` , `app/sitemap.ts` , `app/opengraph-image.tsx` exist. Sitemap only lists `/` , `/privacy` , `/terms` .

Dashboard chrome: `app/dashboard/layout.tsx` (desktop sidebar + mobile FAB). Insights/swaps are **desktop nav**, not mobile bottom nav.

Important web components

Component	Path	Purpose	Used by	
AuthProvider	<code>components/AuthProvider.tsx</code>	Load auth + route	<code>app/layout.tsx</code>	Wc
AuthForm / Navbar	<code>components/ui/navbar.tsx</code>	Landing auth	<code>app/page.tsx</code>	Wc mc bu !ha
SubscriptionGate	<code>components/SubscriptionGate.tsx</code>	Trial overlay	dashboard layout	UI- en
SubscriptionUpsell	<code>components/SubscriptionUpsell.tsx</code>	Coupon / WhatsApp CTA	onboarding	Co rea Wh a I no
FlatSwitcher	<code>components/FlatSwitcher.tsx</code>	Multi-flat	dashboard layout	Re Fir pro
GoingOutModal	<code>components/GoingOutModal.tsx</code>	OOS + swaps	dashboard/members	Re
NPSBanner	<code>components/NPSBanner.tsx</code>	NPS	dashboard home	Re NF
NotificationToast	<code>components/NotificationToast.tsx</code>	In-app toasts	layout	Cli to: pu
PWA*	<code>PWAInstallPrompt</code> , <code>ServiceWorkerRegistration</code> , <code>contexts/PWAContext.tsx</code>	Install	root layout	Pre
HeroCanvas	<code>components/HeroCanvas.tsx</code>	Three.js	Usage not fully confirmed	Po un
shadcn-style	<code>components/ui/{button,card,input,label}.tsx</code>	Primitives	dashboard	Ce
Landing extras	<code>animated-tabs</code> , <code>container-scroll-animation</code> , <code>testimonial-slider</code>	Marketing	landing	Ma on

Android screens (from `HabitiqApp.kt` + `AppShell.kt`)

Routes: `login`, `signup`, `intent_chooser`, `main`, `create_flat`, `create_flat_brilliant`, `join_flat`, `settings`. Main tabs: **Home**, **Discover**, **Manage**, **Profile** plus `+` sheet. Discover is first-class on Android (`DiscoverBoardScreen`, `UseAFlatDiscover`, `FindFlatmateDiscover`, chat via `MessagingRepository`).

UI / design system

- **Web tokens:** `app/globals.css` (`@theme` indigo brand + amber accent, light/dark). Landing / **hardcodes** its own dark palette (`#08080C`, etc.) — **not** the dashboard tokens.
- **Android:** `ui/theme/Theme.kt`, `BrandColors.kt`, `FigmaColors.kt`, `components/Hq*`.
- Dark mode: `localStorage` `habitiq-theme` in `app/layout.tsx` + `settings/profile` toggles.
- Responsive: `md`: sidebar vs bottom nav.
- Charts: **no chart library**; Insights is custom calendar/grids.
- Animations: Framer Motion (nav FAB, landing).

4. Backend audit

Framework. None as a server app. **Firestore** from the client. Next.js is the web host.

Concern	Finding
API architecture	Direct collection paths <code>users/</code> , <code>flats/{id}/...</code>
Controllers	N/A
Services	<code>lib/flatService.ts</code> , <code>couponService.ts</code> , <code>npsService.ts</code> , store methods
Server actions	Not used for domain data (client 'use client' pages)
Jobs / cron / queues / workers	Not in repo
Webhooks	Not in repo
Validation	Mostly client; rules constrain some fields (e.g. <code>expense createdBy</code> , <code>coupon usedBy</code>)
Error handling	<code>try/catch</code> + <code>alert</code> in auth; transaction in <code>joinFlat</code>
Logging	<code>console.error</code> / Android analytics events
Caching	Zustand persist for auth identity (not task data)
Rate limiting	None in app code

Notable operations (not HTTP)

Examples from `lib/flatService.ts` / store:

- `createFlat` — writes `flats/{id}`, member, `users/{uid}`; 30-day `trialEndDate`.
- `joinFlat` — transaction + 8-member cap via `memberCount`.
- `requestToJoinFlat` / approve / reject — `joinRequests`.
- Task complete — `rotationEngine.completeTask` then `setDoc` task + activity + **broken** `addBehavioralEvent` (missing module).

- Coupons — read `coupons/{CODE}` , update flat + `usedBy` (**not a transaction**).

Android mirrors via `*Repository.kt` and `AuthRepository.kt` (`sendPasswordResetEmail` present).

Known limitation: `store/useFlatStore.ts` imports `@/lib/behavioralEvents` and `@/lib/discoveryTypes` .

Those files are not in `lib/` . This is a **compile-breaking** web issue until restored or removed.

5. Database blueprint

Tech: Cloud Firestore (no SQL, no Prisma). **ORM:** Firebase SDK.

Schema location: implied by `firestore.rules` + TypeScript interfaces in `store/useFlatStore.ts` + Android models.

Indexes: `firestore.indexes.json` — **empty**. Queries using `orderBy` **will need composite/single-field indexes in the Firebase project**; cannot tell if production Console already has them.

Collections (from rules + writes)

```
users/{uid}
  blocked/{targetId}          # Discover blocks (rules); web usage not found
flats/{flatId}                # name, adminUid, memberCount, joinMode, subscription*, vacancy
  members/{uid}
  tasks/{taskId}
  activityLog/{id}
  npsResponses/{id}
  swapRequests/{id}
  expenses/{id}
  settlements/{id}
  recurringBills/{id}
  billInstances/{id}
  monthCycles/{id}           # delete forbidden
  joinRequests/{id}
coupons/{code}                # Console-created; client redeem
seekerProfiles/{profileId}    # Android Discover
messages/{messageId}          # Android chat
discoveryConnections/{connId}
discoveryReports/{reportId}
```

Relationships (actual)

```
User (users/{uid}: activeFlatId, flatIds[], email, fcmToken on Android)
  → Flat (flats/{id})
    → Members, Tasks, Expenses, Settlements, Bills, Cycles, Swaps, Activity, JoinRequests,
NPS
  → Coupons (flatId in usedBy)
  → SeekerProfile / Messages / Connections (Discover, Android)
```

PK: document IDs (`FLAT-xxxx` , `uid, uid`).

FK: fields like `paidBy` , `currentAssignedUserId` — **not DB-enforced**.

Enums (app-level): member `role` / `status` , task `frequency` / `status` , expense categories, bill instance status, subscription `trial|active|expired` .

Migrations: ad hoc in `getUserFlatProfile` (old `flatId` → `activeFlatId` + `flatIds`).

Seed: mock constants in the store, not a Firestore seed script.

Unused vs missing:

- Rules for Discover **used by Android, not by web UI**.
- Web **references** `VacancyListing` / `addBehavioralEvent` **without files**.
- `firestore.rules` allow `users/{id}/blocked` — no web implementation found.
- **Cannot determine** unused collections in the live Firebase project from the repo alone.

6. Authentication & authorization

Capability	Web	Android
Login Google	<code>signInWithPopup</code> / <code>signInWithRedirect</code>	Google Sign-In (<code>android/README.md</code>)
Email/password	<code>signInWithEmailAndPassword</code> / <code>createUserWithEmailAndPassword</code>	Login/Signup screens
Logout	<code>firebaseSignOut</code> + clear store	<code>AuthRepository.signOut</code>
Session	Firebase Auth + Zustand persist <code>habitiq-auth</code>	Firebase Auth
JWT/cookies	Firebase ID tokens (SDK)	Same
Password hashing	Firebase Auth (not app code)	Same
Password reset	Not found	<code>sendPasswordResetEmail</code>
Email verification	Not found	Not found in grep
Mock login	<code>loginAsAdminMock</code> / Member uid <code>u1/u2</code>	N/A
Roles	<code>members.role</code> <code>admin</code> <code>member</code>	Same
Org/teams	Flats , not orgs	Same
Protected routes	Client <code>AuthProvider</code> only — pages are not server-protected	Nav graph
Backend AuthZ	<code>firestore.rules</code>	Same project

Journey: Signup/login → `onAuthStateChanged` → `users/{uid}` profile → if no `activeFlatId` onboarding/intent chooser → dashboard/main → rules on every write → logout.

UI vs real: Mock buttons only when `!hasKeys` . Trial lock is **UI**. Admin-only pages (manage-flat) check `role` in React; **URL is still fetchable**; rules still apply to writes.

7. API & data flow (major features)

Create flat: Onboarding → `createFlat` → Firestore `flats` + `members` + `users` → `addFlatToState` → `initFirestoreListeners` .

Join: Invite code → `flatExists` / `getFlatJoinMode` → `joinFlat` or `requestToJoinFlat` → listeners.

Complete task: Dashboard → `markTaskCompleted` → `completeTask()` → `setDoc` task → `addActivity` → attempted behavioral event.

Expense: Expenses page → `addExpense` → `flats/.../expenses` → `computeBalances` in `lib/expenseUtils.ts` (client) → dashboard cards.

Bill generate: Admin → `generateBill` → `billInstances` (+ activity).

Swap: `createSwapRequest` → recipient `resolve` (rules: only `toUserId` may accept/reject).

Coupon: Gate/upsell → `validateAndRedeemCoupon` → `flats` + `coupons` .

Discover (Android): `DiscoveryRepository.observeActiveVacancies` reads **flat docs' vacancy**; chat via `messages` . **Web:** `updateVacancy` in store writes `flats.vacancy` but **no Discover browse UI**.

8. Third-party integrations

Provider	Purpose	Where	Env	Status
Firebase Auth	Identity	<code>lib/firebase.ts</code> , Android Auth	<code>NEXT_PUBLIC_FIREBASE_*</code>	Used
Cloud Firestore	Data	same	same	Used
Firebase Analytics	Web lazy <code>getAnalytics</code> ; Android <code>AppAnalytics.kt</code>		<code>MEASUREMENT_ID</code>	Partial
Google OAuth	Sign-in	Auth stores; <code>next.config.ts</code> rewrite to <code>garbage-f79f7.firebaseio.com</code>	Authorized domains	Used if Console configured
FCM	Push	<code>MainActivity.registerFcmToken</code> , <code>HabitiqFcmService.kt</code>	<code>google-services</code>	Token saved ; no send Functions in repo
Unsplash	<code>next.config.ts</code> <code>images.remotePatterns</code>		—	Image host allowlist; usage not

				fully traced
Stripe / Razorpay / WhatsApp / OpenAI	—	Mentioned in <code>PRODUCT.md</code> only	—	Missing

9. Environment variables

Secret **values** are not listed.

Variable	Purpose	Used by	Required?	Cc
<code>NEXT_PUBLIC_FIREBASE_API_KEY</code>	Firebase web key	<code>lib/firebase.ts</code>	For real mode	Temp .env. Live v inspe
<code>NEXT_PUBLIC_FIREBASE_AUTH_DOMAIN</code>	Auth domain (default <code>habitiq.app</code>)	same	Recommended	Exam
<code>NEXT_PUBLIC_FIREBASE_PROJECT_ID</code>	Project	same	Yes (real)	Exam
<code>NEXT_PUBLIC_FIREBASE_STORAGE_BUCKET</code>	Storage (init only)	same	For Storage	Exam Stora foun
<code>NEXT_PUBLIC_FIREBASE_MESSAGING_SENDER_ID</code>	FCM web	same	If messaging	Exam
<code>NEXT_PUBLIC_FIREBASE_APP_ID</code>	App id	same	Yes (real)	Exam
<code>NEXT_PUBLIC_FIREBASE_MEASUREMENT_ID</code>	GA	same	Optional	Exam
CI	Playwright workers/retries	<code>playwright.config.ts</code>	No	CI en

Android uses `google-services.json` (not a Next env var). Files `.env.prod.check` / `.env.vercel.check` exist; **contents not read**.

10. File & folder architecture

C:\garbage\	
├ app/	Next App Router
├ components/	Shared + landing UI
├ store/	Zustand
├ lib/	Firebase, flats, rotation, money, NPS, coupons
├ hooks/	useSubscription

- | contexts/ PWA
- | tests/ Playwright
- | public/ sw.js, manifest, assets
- | android/ Native app + unit tests
- | project_1/ Obsidian product docs (not runtime)
- | design-system/ Design markdown
- | releases/ QA notes / APK checklist / this audit
- | firestore.rules, firestore.indexes.json, firebase.json
- | next.config.ts, middleware.ts, netlify.toml
- | patch-*.js, fix-*.js # one-off scripts – not part of runtime

Dead / temporary: app/(auth)/login/page.tsx ; patch-expense.js , patch-modal.js , patch-return.js , fix-encoding.js , fix-encoding2.js , fix-currency.js . **Duplicate:** settings vs profile.

11. Feature-by-feature status

Feature	FE	BE	DB	API	Integration	Status
Rotation + overdue	Yes	Client	tasks	SDK	—	Fully implemented
Swaps	Yes	Client + rules	swapRequests	SDK	—	Fully implemented
Expenses/settlements	Yes	Client + rules	yes	SDK	—	Fully implemented
Recurring bills	Yes	Client + rules	yes	SDK	—	Fully implemented
Multi-flat	Yes	users.flatIds	yes	SDK	—	Implemented
Join approval	Yes	joinRequests	yes	SDK	—	Implemented
Google/email auth	Yes	Firebase	users	Auth	Google	Partial (no web reset/verify)
Trial/coupons	Web UI	Client writes	flats/coupons	SDK	—	Partial (not in rules)
PWA/SEO	Web	Next	—	icon route	—	Implemented
Insights	Web	Derived	activity 50	SDK	—	Partial (history truncated)
Discover	Android	Rules + repos	yes	SDK	—	Partial
Web Discover	No	Broken imports	vacancy field	—	—	Missing / broken

Push notify	Android token	No send	fcmToken	—	FCM	Partial
Payments	No	No	No	No	No	Missing
WhatsApp	CTA copy	No	No	No	No	Missing
Admin property dashboard	No	No	No	No	No	Missing (docs only)

12. What is actually working?

If you run `npm run dev` without Firebase keys:

1. Open `/` — **real** landing.
2. Mock Admin/Member — **mocked session**.
3. Dashboard with Sai/Rahul/... — **hardcoded MOCK_***.
4. Completing tasks/expenses — **local Zustand only**, not persisted to a server.
5. Playwright paths match this mode.

If keys are set and Firestore rules/indexes match:

1. Google/email login — **real**.
2. Create/join flat — **real**.
3. Task/expense/bill/swap — **real** listeners.
4. Trial overlay after 30 days — **UI real, writes still allowed** by rules.
5. Discover on **Android** — **likely real** against same DB.
6. Discover on **web** — **not a product surface**; vacancy write may not compile.

Cannot determine from the audit session whether habitiq.app is up or whether Console indexes already exist.

13. What is not working / gaps

Critical

- **Missing** `lib/discoveryTypes.ts` and `lib/behavioralEvents.ts` — imported by `store/useFlatStore.ts`. Impact: web typecheck/build. Next step: restore files or drop imports/ `updateVacancy` / `addBehavioralEvent`.
- **Empty committed indexes** vs multiple `orderBy` listeners. Impact: listeners fail in a fresh Firebase project. Evidence: `firestore.indexes.json`, `useFlatStore.ts` (`activityLog`, `expenses`, `settlements`, `billInstances`, `monthCycles`), Android repos.

High

- Trial/coupon **not enforced in rules**.
- Password reset **web missing**.
- Discover **web missing**; Android-only product split.
- No Cloud Functions for FCM send / billing.
- Auth is **client-router only** (HTML of dashboard still theoretically loadable; data still rule-gated).

Medium

- Orphan routes: activity, settings, manage-flat not in main nav.
- Insights based on **50** activity docs.
- Dual hosting story (`netlify.toml` vs Vercel rewrites vs `middleware` 301).
- Coupon redeem race (`usedBy` two updates).
- Any authenticated user **can read any flat document** (invite lookup) — includes `vacancy` .

Low

- README still mentions Netlify live badge `flatsflow.netlify.app` vs canonical `habitiq.app` .
- `PRODUCT.md` roadmap items marked unchecked that **are** in code (expenses, PWA, multi-flat, leave/kick) — docs stale.
- `Three.js` `HeroCanvas` possibly unused.

14. Code quality

- **Architecture:** Clear for a Firebase SPA; Android is a second client, not a BFF. Dual UI systems (landing vs app vs Compose).
- **Maintainability:** `app/dashboard/expenses/page.tsx` and `useFlatStore.ts` (~1800 lines) are **high-risk concentration**.
- **Types:** `strict: true` but `lib/firebase.ts` uses `any` for `app/auth/db`.
- **Tests:** Playwright mock-only; 4 Android unit tests; **no** web unit tests.
- **A11y:** `tests/accessibility.spec.ts` is shallow (title + tab).
- **Docs:** Lots of markdown in `project_1/` and `PRODUCT.md` **out of date vs code**.
- **Debt:** mock mode, patch JS files, obsolete login, Discover half-ported to web.

15. Security audit (observation only)

- **Client Firebase keys** — expected; documented in `SECURITY.md` .
- **No CSP** in `next.config.ts` despite README claiming CSP.
- **X-Frame-Options DENY** with Firebase iframe exception — documented in `next.config.ts` .
- **Trial bypass** — UI-only.
- **Flat document world-readable to any signed-in user**.
- **Coupon docs readable** by any authenticated user (`match /coupons`).
- **seekerProfiles readable** by any authenticated user.
- **Android messages query** orders whole collection then filters client-side — extra reads; still auth-scoped by rules per document.
- **No rate limits** on writes.
- **google-services.json / SHA-1 / web client id** in Android docs — typical for mobile; still sensitive-adjacent.
- This inventory is observational only; nothing was exploited.

16. Testing

Kind	Location	Notes
E2E	<code>tests/*.spec.ts</code> Playwright	Mock mode, no Firebase

Unit web	None	rotationEngine untested in-repo
Unit Android	4 *Test.kt files	Auth/flat helpers
E2E Android	None found	
Coverage	Cannot determine	no coverage config

Before production: compile web; Firebase emulator or staging tests for rules; index creation; auth (Google redirect); join race; trial bypass; Discover abuse (spam listings).

17. Deployment & infrastructure

- **Web host:** `next.config.ts` comments + `PRODUCT.md` point at **Vercel**; `netlify.toml` still present (`@netlify/plugin-nextjs`). `middleware.ts` 301s all non- `habitiq.app` hosts (except localhost).
 - **Build:** `npm run build` (`next build`).
 - **DB:** Firebase project `garbage-f79f7` .
 - **Storage/CDN:** Next/Vercel/Netlify CDN for static; Firebase Storage **configured in env example**, unused in web lib.
 - **CI/CD:** **No** `.github/workflows` .
 - **Docker:** **None** found.
 - **Ready for deploy?** Hosting **shape** yes; **this checkout's web compile is at risk** due to missing modules.
Runtime of production: **Cannot determine from current codebase/session**.
-

18. Dependency audit

From `package.json` :

- **next 16.2.6, react 19.2.4, firebase ^12.13.0, zustand ^5.0.13, framer-motion, lucide-react, tailwindcss 4, three + @react-three/*, clsx, tailwind-merge, tw-animate-css.**
- Dev: **Playwright 1.60, eslint-config-next 16.2.6, typescript 5.**
- **No** `firebase-admin`, `stripe`, `Sentry`, `i18n`.

Deprecated: not proven from lockfile analysis in this pass. **Unused likely:** Three.js stack if `HeroCanvas` unused.

Conflict: none obvious.

Android versions: `android/gradle/libs.versions.toml` (not fully enumerated here).

19. Product blueprint

Confirmed by code: Target = Indian/global flatmates; core loop = duties + money + membership; entities = User, Flat, Member, Task, Expense, Bill, Swap; PWA; Android with Discover and biometric lock (`MainActivity`).

Likely intended (structure, not shipped): In-app Discover chat as Airbnb-like safety (rules + Android); coupons as pre-Stripe monetization; ads/revenue docs in `project_1` — **not implemented as code**.

20. Architecture diagram (actual)

```

USER
├─ Browser (habitiq.app / localhost)
│   └─ Landing + AuthForm
│       └─ Dashboard (tasks, expenses, insights, swaps, members, profile)
│           └─ AuthProvider (client)
└─ Android (habitiq.app)
    └─ Login / Signup / Intent chooser
        └─ Home • Discover • Manage • Profile
            └─ FCM token → users.fcmToken
                └─ Biometric lock (local)

    └─ Firebase JS / Android SDK
        ▼
FIREBASE
├─ Auth (Google, email/password)
├─ Firestore
│   └─ users, flats/*, coupons
│       └─ seekerProfiles, messages, discoveryConnections, discoveryReports
└─ Analytics (optional)

NEXT.JS (web host only)
├─ Security headers + /__auth proxy
├─ Canonical host middleware
└─ GET /api/pwa-icon/[size]

NOT IN REPO
├─ Cloud Functions / queues
├─ Payment provider
└─ WhatsApp Business API

```

21. Current state → target state

Current: Two-client Firebase household app; Android ahead on Discover; web household UI rich; monetization = trial + coupons; tests = mock Playwright.

Target (from this repo's own unfinished surfaces): Compiling web, shared Discover, server-enforced billing, real push, indexes, CI, password-reset parity.

Area	Current	Required	Gap
Web compile	Missing lib modules	Green next build	Restore/remove imports
Indexes	Empty JSON	Match orderBy	Export from Console
Auth	No web reset	Parity	Firebase reset UI
Discover	Android	Both or explicitly Android-only	Product decision + code
Trial	UI	Rules/Functions	Server checks

Push	Token only	Send path	Functions
Payments	Docs	Provider	Not started
CI	None	PR checks	Workflows

22. Development roadmap (from this tree)

Phase 1 — Foundation. Fix missing web modules; decide Discover scope; align docs with code. **Depends on:** nothing. **Outcome:** web builds.

Phase 2 — Core backend. Indexes; emulator tests for rules; optional Functions for coupon atomicity. **Depends on:** Phase 1. **Outcome:** listeners reliable on a new project.

Phase 3 — Core product. Web reset; nav cleanup; activity pagination; Android/web feature parity for household. **Depends on:** stable listeners.

Phase 4 — Integrations. FCM send; then payments; WhatsApp last (docs-only today). **Depends on:** trusted server.

Phase 5 — Analytics. Product metrics beyond GA signup events (`AppAnalytics.kt` is auth/flat only). Insights today ≠ business analytics.

Phase 6 — Security & testing. Trial in rules; Playwright against emulator; expand Android tests.

Phase 7 — Production. Single hosting story; CI; drop dead Netlify/login routes; confirm authorized domains.

23. Final scorecard

Overall project stage: **Functional MVP** (in-flat ops) / **Early MVP** (Discover + billing).

Area	Score
Frontend	72%
Backend	58%
Database	70%
Authentication	68%
API	40%
Integrations	32%
Testing	18%
Security	50%
Deployment	55% (config exists; this tree may not build)

Biggest things already built

1. Smart chore rotation + overdue/OOS skip (`lib/rotationEngine.ts` + store).

2. Full money stack: expenses, settlements, recurring bills, month cycles.
3. Real-time Firestore listeners and membership lifecycle (create/join/kick/leave/transfer).
4. Dual clients on one schema; Android Discover + FCM registration.
5. PWA/SEO/canonical domain + Auth proxy headers.

Biggest missing pieces

1. Web `discoveryTypes` / `behavioralEvents` files.
2. Committed Firestore indexes.
3. Server-side trial/billing and payment provider.
4. Push *sending* and WhatsApp.
5. CI and web password reset.

Biggest technical risks

1. Web does not typecheck as-is.
2. Index gap on a clean Firebase.
3. Client-only authorization for money/trial.
4. God-files (expenses page, flat store).
5. Two hosts + stale docs → wrong production assumptions.

Recommended next 10 actions

1. Restore or delete Discover-related web imports.
2. Run `next build` and fix remaining errors.
3. Create/commit Firestore indexes from the `orderBy` queries.
4. Add web `sendPasswordResetEmail` .
5. Encode trial expiry in rules or Functions.
6. Product-lock Discover: web UI or Android-only.
7. Add GitHub Action: Playwright mock + `tsc` .
8. Implement FCM send in Functions using stored tokens.
9. Remove or quarantine `patch-*.js` / obsolete `/login` .
10. Update `PRODUCT.md` / `README.md` so they match the repo (expenses/PWA already exist).

Closing

A new developer takeaway: **you are building a Firebase-backed household OS with an Android-first Discover layer.** In-flat chores and bills are largely **real**. Discover, payments, and a true API/backend are **not**. The most important fact in *this* checkout is that **the web store already depends on two library files that are not on disk.**